CS 5100 Project Final Paper

Sarah Ruana and Shweta Pattanaik

***Abstract*—In this Battleship, one user is able to play against a trained Monte Carlo AI agent. The user at each turn will be able to see both their and the agent's board, as well as the status of both their and the agents fleet. At the start of the game, the user will pick their name, the size of the board (5-10), and place their ships onto the board. Then, at the start of each turn, each player takes turns guessing the coordinates of the opposing player's ships. The end goal is to find, hit and sink the opponents ships.**

I. Introduction

This project was inspired by fond childhood memories of playing board games, specifically Battleship. As we grow up, the time needed to get the game, find a person to play it with, and actually play it becomes increasingly difficult. As a result, this agent was designed to make effective game play so only one person is required to play. The technical primary objective was to create an artificial intelligence agent that was capable of identifying and sinking opponent ships in the least number of moves. This was done by using probabilistic search strategies, heuristic-based decision-making, and Monte Carlo simulation techniques.

The final framework includes a fully functional game environment consisting of defined rules, board representations, player interactions, and win/loss conditions. The agent's performance was evaluated using metrics compared against a randomized agent that simply chose a cell with no rhyme or reason. The primary metrics that were compared were average shots/hits, as well as shots tracked over time.

Theoretically, this project was inspired by Monte Carlo search methods that utilized probability grid maps whilst being continuously updated to be flexible. Chung and Burdick's paper on search decision using probabilistic search strategies provided the strong conceptual basis for our approach to this problem. In the paper, they proposed "investigative search strategies, which are computationally inexpensive and, as a result, are able to leverage updated information that is obtained from the searcher's observations to adaptively compute effective search trajectories" (Chung & Burdick, 2012). This very closely parallels our Battleship problem as each grid cell represents a potential ship location which gets updated after each turn. Thus, feedback from guesses (hit/miss) serve as observational data for the model.

A key contribution of the paper was its use of a unified Bayesian decision model to represent and update beliefs about the target's potential location. Each cell in the search space was assigned a probability representing the likelihood of containing the target using simulations to model potential ship placements. These probabilities were updated sequentially as new observations are made. Furthermore, decision thresholds were used to update the probability of a cell when it was deemed impossible given information we already know.

Additionally, the paper explored how probabilistic belief maps could continuously be updated even as the

observations of the board state change. The authors note that the searcher's objective was to determine "the presence (and cell location) or the absence of a stationary object in A in minimum (expected) time" (Chung & Burdick, 2012). They did this by using a Monte Carlo simulation to gather the probability distribution across thousands of trials.

Then, the paper concludes using results from their Monte Carlo simulations that adaptive strategies significantly outperform non-adaptive approaches. This translates to our project in two ways. First, we show that our model outperforms a random agent that attacks cells at random. This shows how an adaptive approach that uses the information we know provides for a better and harder game play for the user. Secondly, within the agent itself, the amount of simulations is adaptive as game play goes on. At the start of the game, there are many potential ship locations so doing a very large number of simulations is required in order to accurately model the probabilities for each cell. However, by the end of the game, there are very few valid locations available and not as many simulations are needed.

By integrating these concepts, our project aims to bridge theoretical models of probabilistic search with both an interactive and practical game environment. Overall, our AI agent uses Monte Carlo simulations to build a probability map, then applies a filtering heuristic to eliminate impossible cells before choosing the best cell to target.

Beyond gameplay, this project highlights how the use of artificial intelligence techniques in game play can be generalized to real-world uncertain applications. For example, search-and-rescue operations where determining the location of people under a pile of rubble becomes a necessity when the amount of rescuers is limited. Additionally, Monte Carlo methods can be applied to resource allocation by simulating the outcomes of each allotment. This work demonstrates combining probabilistic reasoning and adaptive strategies to solving complex search problems.

II. Methods

Battleship was approached in a very similar fashion. This project was implemented in Python and managed through Github. The structure used a Model-View-Controller (MVC) architecture that separated all three components based off of game logic, user interaction, input handling, and testing. The Model layer consisted of five core classes: Ship, Board, Player, AIPlayer, and finally the BattleshipGame itself. The Ship class tracked an individual ship representing its name, size, positions, as well as its contained hits. The functionality for this class determines if a ship has been sunk, if a hit is registered to one of its positions, and a string representation for the class. The Board class represented an individual player's board with its grid state as well as attack logic. Functionally, this also handles legality with attacking a coordinate, noting what to do if the coordinate is unoccupied, already attacked, etc. The Player and AIPlayer class represent a human player and their name, as well as their board and fleet. To extend this class, the AIPlayer class automatically places its fleet and handles an automatic attack selection using the Monte Carlo agent. Finally, game model resides in the Battleship Game class which essentially acts as the game for both the users. It encapsulated all core game mechanics, including starting/ending the game, enforcing rules (must

place ships before starting the game, cannot place ships after starting the game), and processing attacks This class also contains a turn-based progression to alternate the moves for each player correctly. Additionally, it also continuously evaluates win conditions by checking whether all ships in a player's fleet have been sunk, at which point it declares the winner and terminates the game.

As for the two smaller portions of the MVC component, the view deals with a textual interface for the game solely responsible for representing the current game state to the user. It lays out both the users board, fleet, opponents fleet, and a representation of the opponents board (note: the ships on the user's board are shown, the ships on the opponent's board are hidden). It also shows the result of each attack for both the user and the agent, and the winner at the end of the game. Next, the controller connects both the view and the model by relaying information from the view (user/agent) to the model, and then from the model back to the view to update it. In this class, it checks the validity of each input, as well as displays prompts to the users for certain actions. Some examples include recognizing the turn of the ai agent by pressing enter, confirming the user's turn by pressing enter, or displaying the winner.

The core of the project was the MonteCarlo agent class (located in the model). This class (under normal gameplay, not testing) is the designated opponent to the user. For each of the model's turns, the agent received both the user's board state, and a list of their unsunk ships. To determine the amount of simulations needed to run each turn, the agent scaled it based on how many valid cells to attack remained. This made it adaptive to the current state of the game. It then ran the simulations (capped between 500 and 3000) to build a probability map that determined the best possible attack location for the agent to take. For each simulation the agent would randomly place ships on a copy of the board, and build a proposed map of the board. A simulation was thrown out if it produced an invalid state. This means that the proposed map physically could not be possible (e.g. ships couldn't overlap other ships, ships couldn't be placed on known misses, and known hits had to be covered by a ship). The agent then counted the probability that a cell was occupied over all plausible simulations to produce an individual probability for each cell. To enhance the model, the agent then filtered the map to zero out certain cells where the smallest unsunk ship could not physically fit both horizontally and vertically. Finally, to make its choice, the agent picks the most probable cell to attack.

There were multiple ways that we tested the correctness of our implementation. To start, each core class was tested on general correctness that covered initialization, edge cases, and integration between the components using Python's built-in unittest framework. These included for example, correct ship placement, boundary constraints, etc. These tests were designed to ensure that both components behave as expected under normal and exceptional conditions, providing confidence in the reliability and functionality of the system.

The next set of testing which will be explained more in depth in the results section, was specifically created to test the Monte Carlo Agent. The evaluation folder contains all of the required classes necessary. First, there is the simple agent (RandomAgent) that just picks any valid cell to target regardless of any previous known

information about the grid. The class AgentComparison utilizes both agents to essentially play against each other. It runs 500 times to produce various different graphs to represent the validity of the Monte Carlo agent. Finally, the runComparison class is just used to run the AgentComparison class and show the results from the terminal.

III. Results

To evaluate the performance of the Monte Carlo agent, we ran 500 simulated games where the two agents (Monte Carlo and Random) played against each other. Over the trials, we tracked the average shots to win, the number of shots to win distribution, shots per game over time, and average hit rate. The results showed that the Monte Carlo agent had a clear advantage over a simple random agent. For example, the Monte Carlo agent won games in $\sim$66 shots, while it took the random agent $\sim$95 shots (Figure 1).

Figure 2 (number of shots to win) shows how the Monte Carlo agent's games were spread fairly evenly between 30 and 80 shots. On the contrary, the random agent was very concentrated around 90 to 100 shots. This makes sense because the random agent has no targeting logic and so it needs to just systematically cover more of the grid compared to the Monte Carlo agent. The Monte Carlo agent's wider and lower distribution reflects the benefit of using a probability map that updates after each turn to guide attacks rather than treating each cell as a uniform probability.

The shots-per-game graph (Figure 3) shows the pattern described in Figure 2 except spread over all 500 games. Consistently, the Monte Carlo's blue line stays below the random agent's red line. Even though there is some variance between the individual games, the trend for the Monte Carlo agent is stable, demonstrating that it is not reliant on luck.

Finally, as demonstrated in Figure 4 (average hit rate) the Monte Carlo agent's average hit rate was roughly 10% higher than the random agent. This directly reflects the importance of the probability map influencing where the Monte Carlo agent is firing. The Monte Carlo agent's probability map allows it to fire at cells where there is most likely to be a ship given what we know about the board. For example, the Monte Carlo agent utilizes filtering the probability map to zero out cells that are impossible given the cell and the smallest remaining ship. This allows the Monte Carlo agent to not guess blindly like the random agent. Over time, this difference compounds since a higher hit rate means fewer wasted moves and explains the difference in the average shots needed to win a game.

Overall, the four figures prove that the Monte Carlo simulation approach to Battleship significantly outperforms a simple agent which does not account for updates to the game state. The biggest gains came from the Monte Carlo agent to concentrate on cells with a high probability and avoid impossible locations, especially as the game progressed and more information became available.

IV. Conclusions

All in all, this project successfully demonstrated that a Monte Carlo simulation-based agent can outperform a simple random strategy based agent in Battleship and prove a decent opponent to a human player. The results in Figures 1 through 4 confirm that both probabilistic reasoning and adaptive decision making can produce a

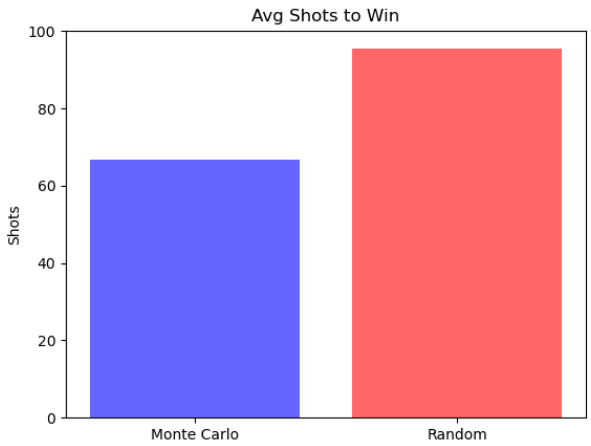

Fig. 1. Avg Shots to Win

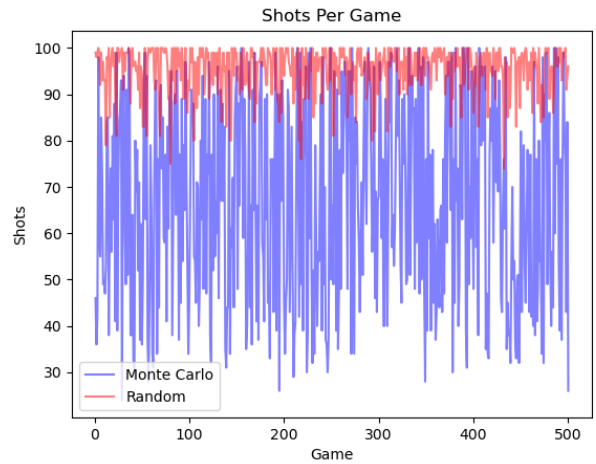

Fig. 3. Shots Per Game

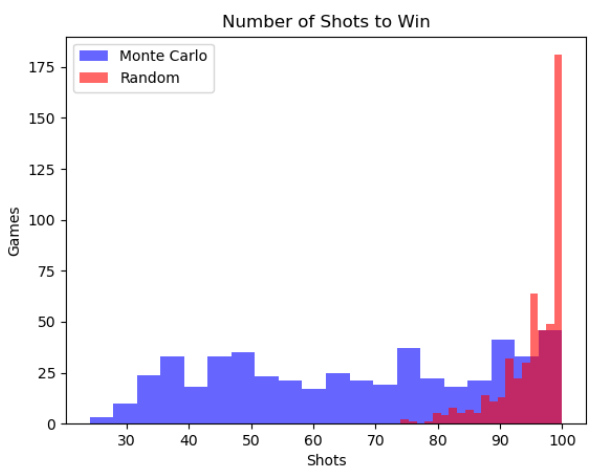

Fig. 2. Number of Shots to Win

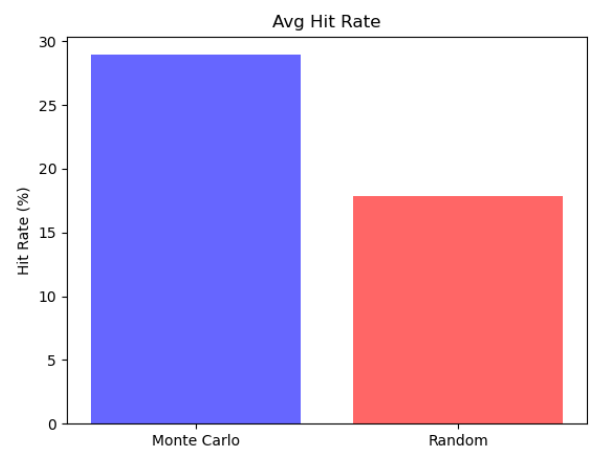

Fig. 4. Avg Hit Rate

consistent and measurable advantage. The agent had a complex ability to build and update probability maps, scale simulations based on the current game state and filter out physically impossible locations.

The biggest improvement, aside from improving the agent, that could be added to the game is to make the user experience better. The primary way to do this would be implementing a GUI environment where the player could click directly on the board to place their ships and to select an attack. This would make the game more interactive and make the current game state clearer for the user. However, it is important to note that the current view (Figure 5) functionally works and displays everything that the user could need to see such as current board states, as well as fleet statuses.

Aside from the general interface for the user, the other area where we could have improved was simply on the agent itself. The agent was tested against a simple

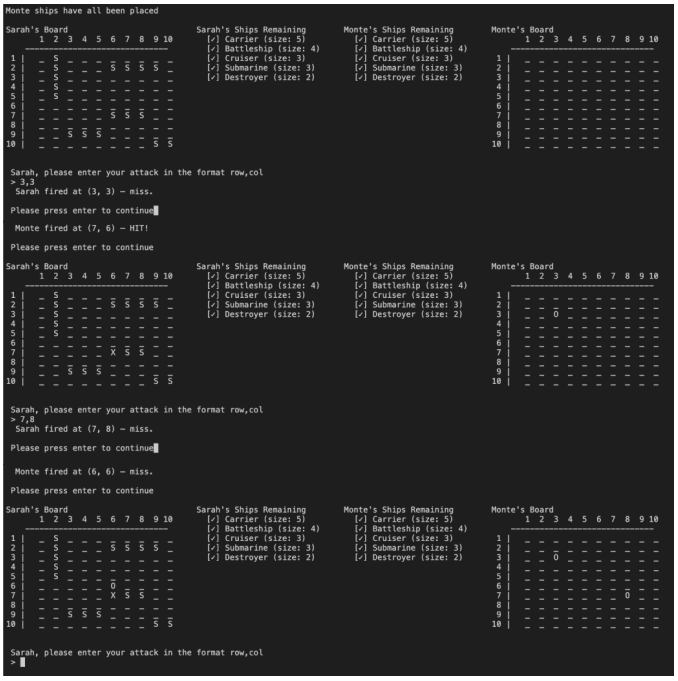

Fig. 5. This is a sample code output on the terminal of both user/agent setting up ships, and the user board results from the agent making both a miss and a hit (note: when actually playing, the output clears at each turn to make readability clearer)

random agent but it would have been interesting to test it against other agents as well. For example, we could have tested it on another Monte Carlo agent that does not apply a filter to zero out impossible cells. This simply would just expose some weaknesses from our original approach.

To expand further into other approaches, our Monte Carlo simulation sits in between pure heuristic method and full model-based planning. Other approaches could include a hunt-and-target model that alternates between randomly searching and focusing on cells adjacent to known hits. However, this doesn't take into account the entire board like a Monte Carlo agent. Another approach is described by Aydin Schwartz in his Towards Data Science blog post, modeled after Nick Berry's DataGenetics algorithm blog post. This agent combines many different pathways. First, it is able to eliminate at least half of the cells by only firing on cells whose row and column has to have a ship in it. It then added a probabilistic heat strategy to target cells closer to the center of the board. This gets recalculated after known misses and hits. This is a similar board pattern to our Monte Carlo agent where it will make an assumption about the board and then apply a probabilistic search method to determine the most likely cell. It would be interesting to see where the two agents would fall on our metric testing.

Ultimately, this project demonstrated how classical AI techniques like probabilistic search and Monte Carlo simulations can be applied to a simple but clear game environment to produce an intelligent agent capable of playing against a user. Even with incomplete information, an agent that is able to reason about what is and is not possible will outperform an agent that does not.

References

[1] T. H. Chung and J. W. Burdick, "Analysis of search decision making using probabilistic search strategies," *IEEE Transactions on Robotics*, 2012.

[2] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in science & engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[3] Wikipedia. (2026) Battleship (game). [Online]. Available: https://en.wikipedia.org/wiki/Battleship_(game)

[4] A. Schwartz. (2022) Coding an intelligent battleship agent. [Online]. Available: https://towardsdatascience.com/coding-an-intelligent-battleship-agent-bf0064a4b319

[5] N. Berry. (2011) Battleship. [Online]. Available: http://www.datagenetics.com/blog/december32011/